

```
# 1. Import Required Libraries
# =====
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')

# Models & Tools
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import (
    RandomForestClassifier, AdaBoostClassifier, GradientBoostingClassifier, ExtraTreesClassifier
)
from xgboost import XGBClassifier
from lightgbm import LGBMClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# reading dataset
chronic_df = pd.read_csv('/content/kidney_disease.csv')
chronic_df.head()
```



	id	age	bp	sg	al	su	rbc	pc	pcc	ba	...	pcv	wc	rc	htn	dm	cad	appet	pe	ane	classification
0	0	48.0	80.0	1.020	1.0	0.0	NaN	normal	notpresent	notpresent	...	44	7800	5.2	yes	yes	no	good	no	no	ck
1	1	7.0	50.0	1.020	4.0	0.0	NaN	normal	notpresent	notpresent	...	38	6000	NaN	no	no	no	good	no	no	ck
2	2	62.0	80.0	1.010	2.0	3.0	normal	normal	notpresent	notpresent	...	31	7500	NaN	no	yes	no	poor	no	yes	ck
3	3	48.0	70.0	1.005	4.0	0.0	normal	abnormal	present	notpresent	...	32	6700	3.9	yes	no	no	poor	yes	yes	ck
4	4	51.0	80.0	1.010	2.0	0.0	normal	normal	notpresent	notpresent	...	35	7300	4.6	no	no	no	good	no	no	ck

5 rows × 26 columns

Dataset Description

- age - age
- bp - blood pressure
- sg - specific gravity
- al - albumin
- su - sugar
- rbc - red blood cells
- pc - pus cell
- pcc - pus cell clumps
- ba - bacteria
- bgr - blood glucose random
- bu - blood urea
- sc - serum creatinine
- sod - sodium
- pot - potassium
- hemo - hemoglobin
- pcv - packed cell volume
- wc - white blood cell count
- rc - red blood cell count
- htn - hypertension
- dm - diabetes mellitus

- cad - coronary artery disease
- appet - appetite
- pe - pedal edema
- ane - anemia
- class - class
- ##### %
- Number of Attributes: 24 + class = 25 (11 numeric ,14 nominal) %

Attribute Information :

- 1.Age(numerical)

age in years

- 2.Blood Pressure(numerical)

bp in mm/Hg

- 3.Specific Gravity(nominal)

sg - (1.005,1.010,1.015,1.020,1.025)

- 4.Albumin(nominal)

al - (0,1,2,3,4,5)

- 5.Sugar(nominal)

su - (0,1,2,3,4,5)

- 6.Red Blood Cells(nominal)

rbc - (normal,abnormal)

- 7.Pus Cell (nominal)

pc - (normal,abnormal)

- 8.Pus Cell clumps(nominal)

pcc - (present,notpresent)

- 9.Bacteria(nominal)

ba - (present,notpresent)

- 10.Blood Glucose Random(numerical)

bgr in mgs/dl

- 11.Blood Urea(numerical)

bu in mgs/dl

- 12.Serum Creatinine(numerical)

sc in mgs/dl

- 13.Sodium(numerical)

```
sod in mEq/L
```

- 14.Potassium(numerical)

```
pot in mEq/L
```

- 15.Hemoglobin(numerical)

```
hemo in gms
```

- 16.Packed Cell Volume(numerical)
- 17.White Blood Cell Count(numerical)

```
wc in cells/cumm
```

- 18.Red Blood Cell Count(numerical)

```
rc in millions/cmm
```

- 19.Hypertension(nominal)

```
htn - (yes,no)
```

- 20.Diabetes Mellitus(nominal)

```
dm - (yes,no)
```

- 21.Coronary Artery Disease(nominal)

```
cad - (yes,no)
```

- 22.Appetite(nominal) appet - (good,poor)
- 23.Pedal Edema(nominal)

```
pe - (yes,no)
```

- 24.Anemia(nominal)

```
ane - (yes,no)
```

- 25.Class (nominal)

```
class - (ckd,notckd)
```

```
# 3. Basic EDA & Cleaning
# =====
```

```
print(chronic_df.head())
print("*****")
```

```
print(chronic_df.info())
print("*****")
```

```
print(chronic_df.describe())
```



```

# examine non-null counts by type
# -----
#      id      400 non-null    int64
#      age      391 non-null   float64
#      bp       388 non-null   float64
#      sg       353 non-null   float64
#      al       354 non-null   float64
#      su       351 non-null   float64
#      rbc      248 non-null   object
#      pc       335 non-null   object
#      pcc      396 non-null   object
#      ba       396 non-null   object
#      bgr      356 non-null   float64
#      bu       381 non-null   float64
#      sc       383 non-null   float64
#      sod      313 non-null   float64
#      pot      312 non-null   float64
#      hemo     348 non-null   float64
#      pcv      330 non-null   object
#      wc       295 non-null   object
#      rc       270 non-null   object
#      htn      398 non-null   object
#      dm       398 non-null   object
#      cad      398 non-null   object
#      appet    399 non-null   object
#      pe       399 non-null   object
#      ane      399 non-null   object
#      classification 400 non-null object
dtypes: float64(11), int64(1), object(14)
memory usage: 81.4+ KB
None

```

```

*****
              id      age      bp      sg      al      su  \
count  400.000000  391.000000  388.000000  353.000000  354.000000  351.000000
mean   199.500000   51.483376   76.469072   1.017408   1.016949   0.450142
std    115.614301   17.169714   13.683637   0.005717   1.352679   1.099191
min      0.000000    2.000000   50.000000   1.005000   0.000000   0.000000
25%     99.750000   42.000000   70.000000   1.010000   0.000000   0.000000
50%    199.500000   55.000000   80.000000   1.020000   0.000000   0.000000
75%    299.250000   64.500000   80.000000   1.020000   2.000000   0.000000
max    399.000000   90.000000  180.000000   1.025000   5.000000   5.000000

              bgr      bu      sc      sod      pot      hemo
count  356.000000  381.000000  383.000000  313.000000  312.000000  348.000000
mean   148.036517   57.425722   3.072454  137.528754   4.627244  12.526437
std     79.281714   50.503006   5.741126  10.408752   3.193904   2.912587
min     22.000000   1.500000   0.400000   4.500000   2.500000   3.100000
25%     99.000000   27.000000   0.900000  135.000000   3.800000  10.300000
50%    121.000000   42.000000   1.300000  138.000000   4.400000  12.650000
75%    163.000000   66.000000   2.800000  142.000000   4.900000  15.000000
max    490.000000  391.000000   76.000000  163.000000  47.000000  17.800000

```

▼ Data Cleaning

```

# drop id column
chronic_df = chronic_df.drop('id', axis = 1)

# rename column names to make it more user-friendly

chronic_df.columns = ['age', 'blood_pressure', 'specific_gravity', 'albumin', 'sugar', 'red_blood_cells', 'pus_cell',
                      'pus_cell_clumps', 'bacteria', 'blood_glucose_random', 'blood_urea', 'serum_creatinine', 'sodium',
                      'potassium', 'haemoglobin', 'packed_cell_volume', 'white_blood_cell_count', 'red_blood_cell_count',
                      'hypertension', 'diabetes_mellitus', 'coronary_artery_disease', 'appetite', 'peda_edema',
                      'aanemia', 'class']

chronic_df.head()

```



	age	blood_pressure	specific_gravity	albumin	sugar	red_blood_cells	pus_cell	pus_cell_clumps	bacteria	blood_glucose_random	.
0	48.0	80.0	1.020	1.0	0.0	NaN	normal	notpresent	notpresent	121.0	
1	7.0	50.0	1.020	4.0	0.0	NaN	normal	notpresent	notpresent	NaN	
2	62.0	80.0	1.010	2.0	3.0	normal	normal	notpresent	notpresent	423.0	
3	48.0	70.0	1.005	4.0	0.0	normal	abnormal	present	notpresent	117.0	
4	51.0	80.0	1.010	2.0	0.0	normal	normal	notpresent	notpresent	106.0	

5 rows × 25 columns

✓ According to the data description

- Cols(pcv, wc and rc) needs to convert back in numerical since it is object right now
- Cols(sg, al and su) should be nominal , convert from float to object

```
# Categorical cols like specific_gravity, albumin and sugar which is float type right now
# converting back to nominal data type categorical
chronic_df['specific_gravity'] = chronic_df['specific_gravity'].astype('object')
chronic_df['albumin'] = chronic_df['albumin'].astype('object')
chronic_df['sugar'] = chronic_df['sugar'].astype('object')
```

```
# converting necessary columns like packed_cell_volume, white_blood_cell_count and red_blood_cell_count
# currently it is in object type and converting back to numerical type
chronic_df['packed_cell_volume'] = pd.to_numeric(chronic_df['packed_cell_volume'], errors='coerce')
chronic_df['white_blood_cell_count'] = pd.to_numeric(chronic_df['white_blood_cell_count'], errors='coerce')
chronic_df['red_blood_cell_count'] = pd.to_numeric(chronic_df['red_blood_cell_count'], errors='coerce')
```

```
chronic_df.info()
```



```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 400 entries, 0 to 399
Data columns (total 25 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   age                                    391 non-null    float64
1   blood_pressure                        388 non-null    float64
2   specific_gravity                      353 non-null    object
3   albumin                              354 non-null    object
4   sugar                                 351 non-null    object
5   red_blood_cells                       248 non-null    object
6   pus_cell                             335 non-null    object
7   pus_cell_clumps                       396 non-null    object
8   bacteria                             396 non-null    object
9   blood_glucose_random                  356 non-null    float64
10  blood_urea                           381 non-null    float64
11  serum_creatinine                     383 non-null    float64
12  sodium                               313 non-null    float64
13  potassium                             312 non-null    float64
14  haemoglobin                           348 non-null    float64
15  packed_cell_volume                   329 non-null    float64
16  white_blood_cell_count               294 non-null    float64
17  red_blood_cell_count                 269 non-null    float64
18  hypertension                         398 non-null    object
19  diabetes_mellitus                    398 non-null    object
20  coronary_artery_disease              398 non-null    object
21  appetite                             399 non-null    object
22  pda_edema                            399 non-null    object
23  anemias                              399 non-null    object
24  class                                400 non-null    object
dtypes: float64(11), object(14)
memory usage: 78.3+ KB
```

```
# Extracting categorical and numerical columns
```

```
cat_cols = [col for col in chronic_df.columns if chronic_df[col].dtypes == 'object']
num_cols = [col for col in chronic_df.columns if chronic_df[col].dtypes != 'object']
```

cat_cols

```
['specific_gravity',
 'albumin',
 'sugar',
 'red_blood_cells',
 'pus_cell',
 'pus_cell_clumps',
 'bacteria',
 'hypertension',
 'diabetes_mellitus',
 'coronary_artery_disease',
 'appetite',
 'peda_edema',
 'aanemia',
 'class']
```

by looping & looking at unique values in categorical columns

```
for col in cat_cols:
    print(f"{col} has {chronic_df[col].unique()}values\n")
```

#specific_gravity has [1.02 1.01 1.005 1.015 nan 1.025] values

```
specific_gravity has [1.02 1.01 1.005 1.015 nan 1.025]values
albumin has [1.0 4.0 2.0 3.0 0.0 nan 5.0]values
sugar has [0.0 3.0 4.0 1.0 nan 2.0 5.0]values
red_blood_cells has [nan 'normal' 'abnormal']values
pus_cell has ['normal' 'abnormal' nan]values
pus_cell_clumps has ['notpresent' 'present' nan]values
bacteria has ['notpresent' 'present' nan]values
hypertension has ['yes' 'no' nan]values
diabetes_mellitus has ['yes' 'no' ' yes' '\tno' '\tyes' nan]values
coronary_artery_disease has ['no' 'yes' '\tno' nan]values
appetite has ['good' 'poor' nan]values
peda_edema has ['no' 'yes' nan]values
aanemia has ['no' 'yes' nan]values
class has ['ckd' 'ckd\t' 'notckd']values
```

Start coding or [generate](#) with AI.

```
# replace incorrect values like '\tno', '\tyes', ' yes', '\tno', 'ckd\t', 'notckd' in categorical cols
chronic_df['diabetes_mellitus'].replace(to_replace = {'\tno':'no','\tyes':'yes',' yes':'yes'},inplace=True)
chronic_df['coronary_artery_disease'].replace(to_replace = {'\tno':'no','\tyes':'yes',' yes':'yes'},inplace=True)
chronic_df['class'].replace(to_replace = {'ckd\t':'ckd','notckd\t':'notckd'},inplace=True)
```

```
for col in cat_cols:
    print(f"{col} has {chronic_df[col].unique()}values\n")
```

```
specific_gravity has [1.02 1.01 1.005 1.015 nan 1.025]values
albumin has [1.0 4.0 2.0 3.0 0.0 nan 5.0]values
sugar has [0.0 3.0 4.0 1.0 nan 2.0 5.0]values
red_blood_cells has [nan 'normal' 'abnormal']values
pus_cell has ['normal' 'abnormal' nan]values
pus_cell_clumps has ['notpresent' 'present' nan]values
```

```
bacteria has ['notpresent' 'present' nan]values
hypertension has ['yes' 'no' nan]values
diabetes_mellitus has ['yes' 'no' nan]values
coronary_artery_disease has ['no' 'yes' nan]values
appetite has ['good' 'poor' nan]values
peda_edema has ['no' 'yes' nan]values
aanemia has ['no' 'yes' nan]values
class has ['ckd' 'notckd']values
```

```
# Converting target col class into 0(chronic kidney) and 1(not a chronic kidney)
chronic_df['class'].replace(to_replace = {'ckd':0,'notckd':1},inplace=True)
```

```
# coverting target col into numeric to check correlation
chronic_df['class'] = pd.to_numeric(chronic_df['class'], errors='coerce')
```

```
for col in cat_cols:
    print(f"{col} has {chronic_df[col].unique()}values\n")
```

```
↳ specific_gravity has [1.02 1.01 1.005 1.015 nan 1.025]values
albumin has [1.0 4.0 2.0 3.0 0.0 nan 5.0]values
sugar has [0.0 3.0 4.0 1.0 nan 2.0 5.0]values
red_blood_cells has [nan 'normal' 'abnormal']values
pus_cell has ['normal' 'abnormal' nan]values
pus_cell_clumps has ['notpresent' 'present' nan]values
bacteria has ['notpresent' 'present' nan]values
hypertension has ['yes' 'no' nan]values
diabetes_mellitus has ['yes' 'no' nan]values
coronary_artery_disease has ['no' 'yes' nan]values
appetite has ['good' 'poor' nan]values
peda_edema has ['no' 'yes' nan]values
aanemia has ['no' 'yes' nan]values
class has [0 1]values
```

```
# let's see the cols in numerical col list
for col in num_cols:
    print(f"{col}")
```

```
↳ age
blood_pressure
blood_glucose_random
blood_urea
serum_creatinine
sodium
potassium
haemoglobin
packed_cell_volume
white_blood_cell_count
red_blood_cell_count
```

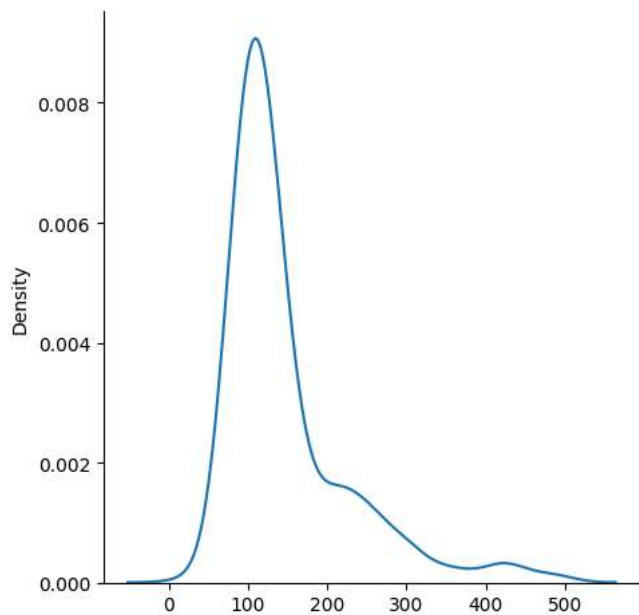
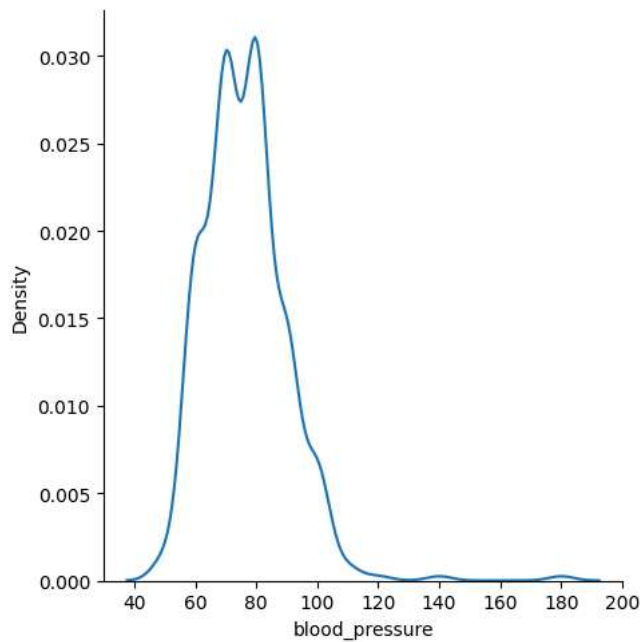
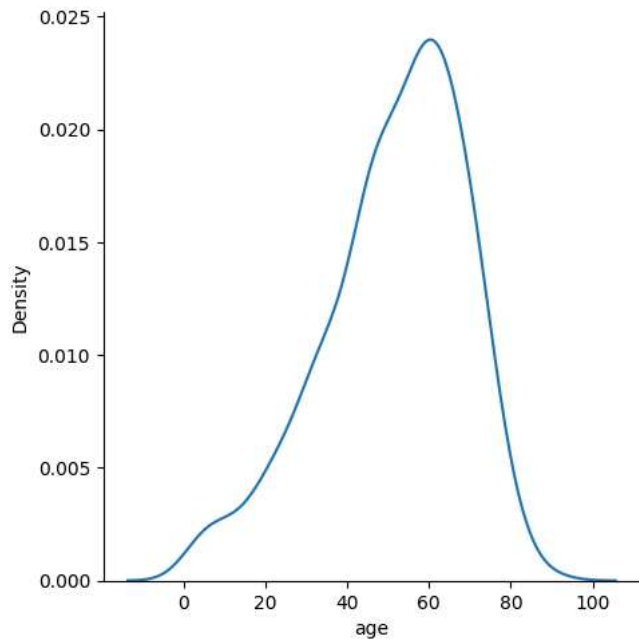
EDA

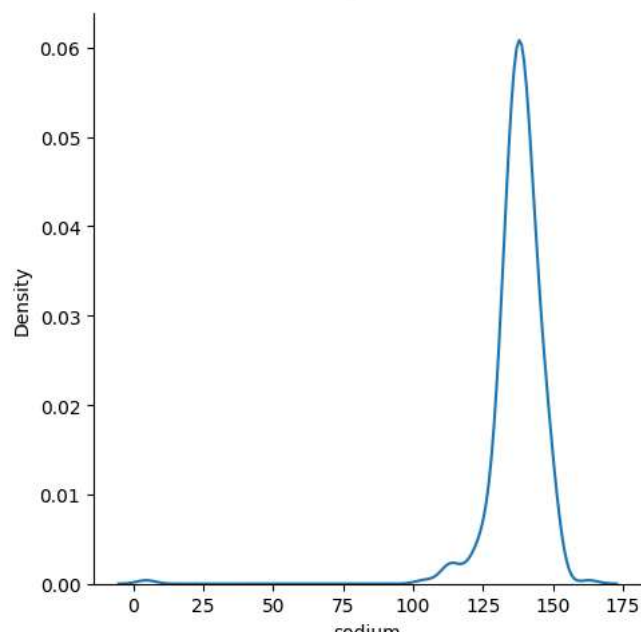
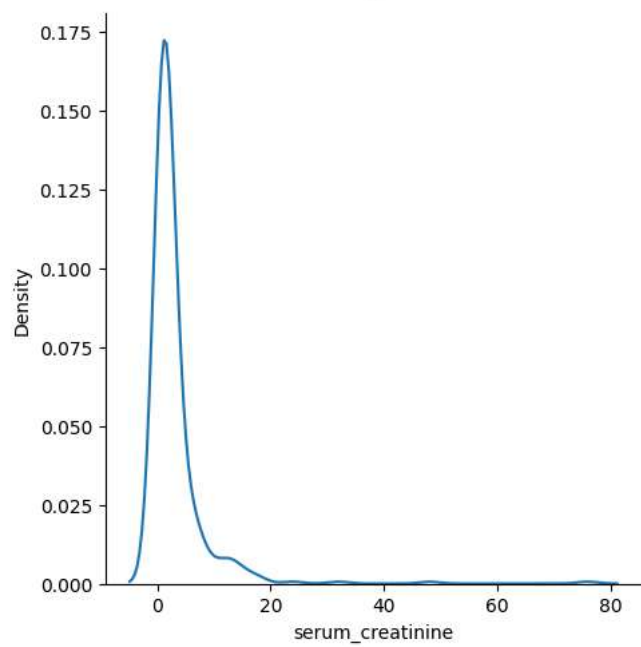
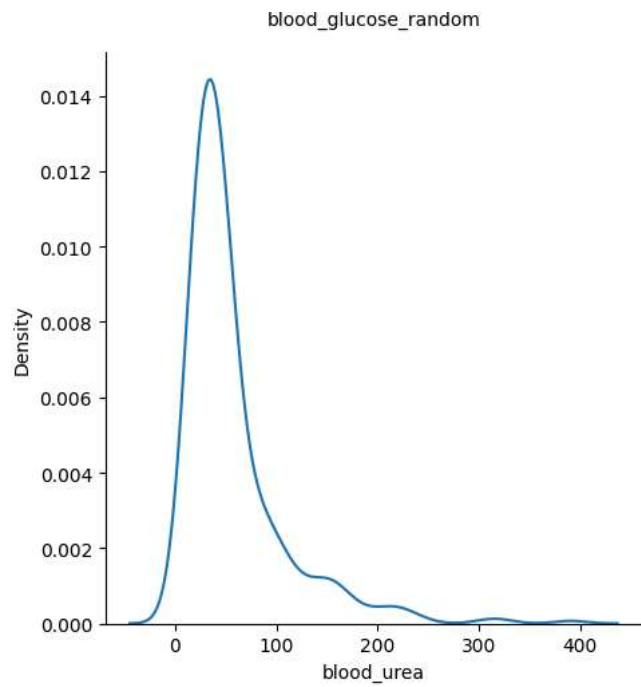
```
# checking numerical features distribution

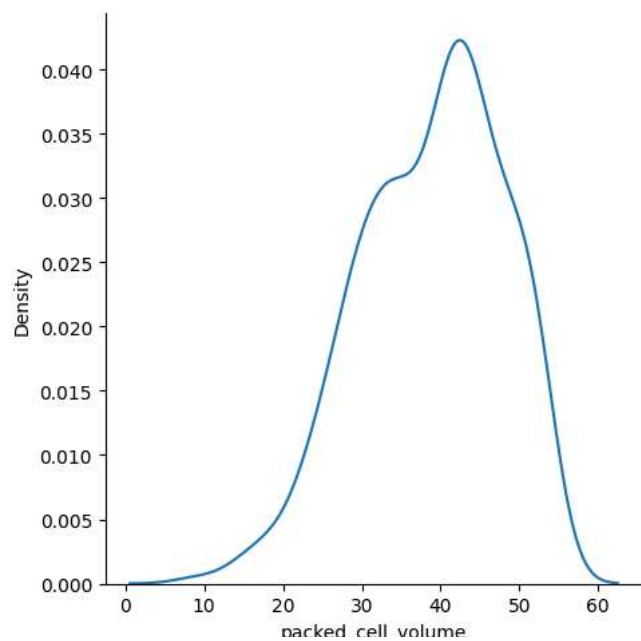
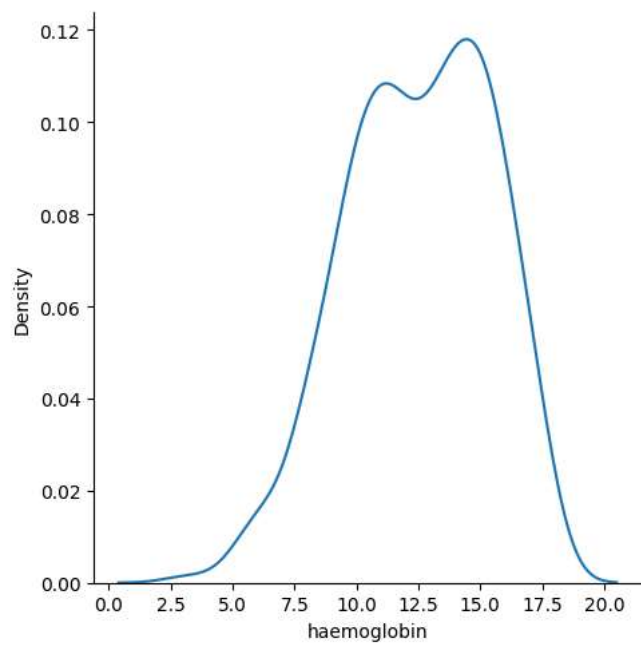
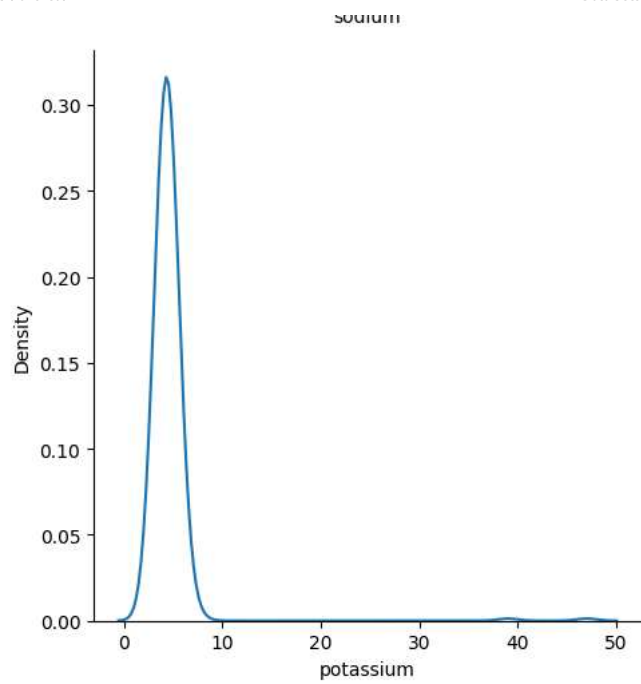
plt.figure(figsize=(20,12))

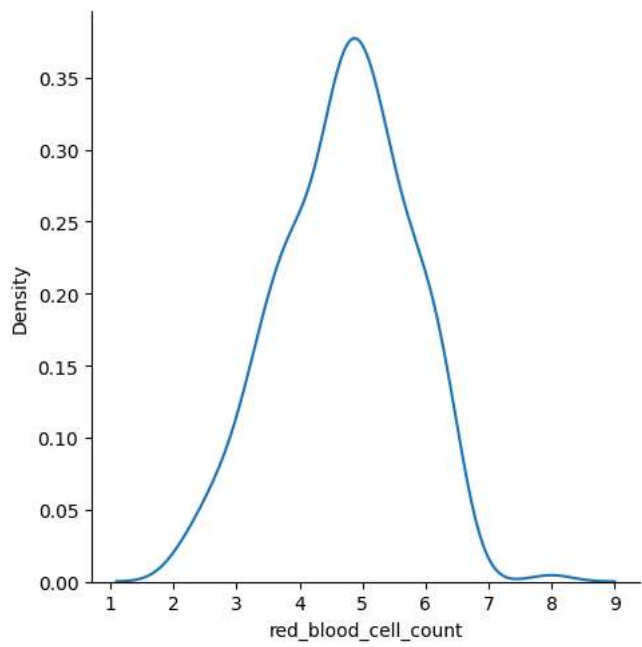
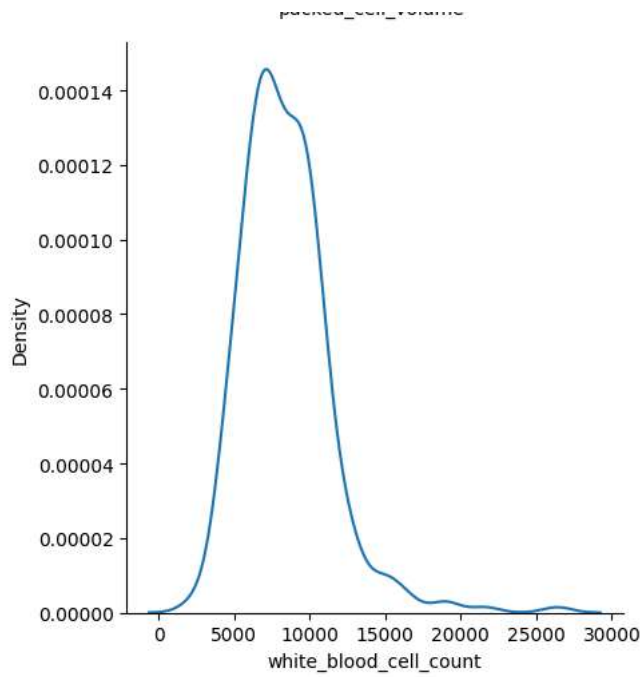
# looping over num cols and checking its distribution
for col in num_cols:
    sns.displot(chronic_df[col],kind='kde')
```


 <Figure size 2000x1200 with 0 Axes>









```
# let's see the cols in cat col list
for col in cat_cols:
    print(f"{col}")
```

```
↗ specific_gravity
  albumin
  sugar
  red_blood_cells
  pus_cell
  pus_cell_clumps
  bacteria
  hypertension
  diabetes_mellitus
  coronary_artery_disease
  appetite
  peda_edema
  aanemia
  class
```

```
# checking cat features distribution
```

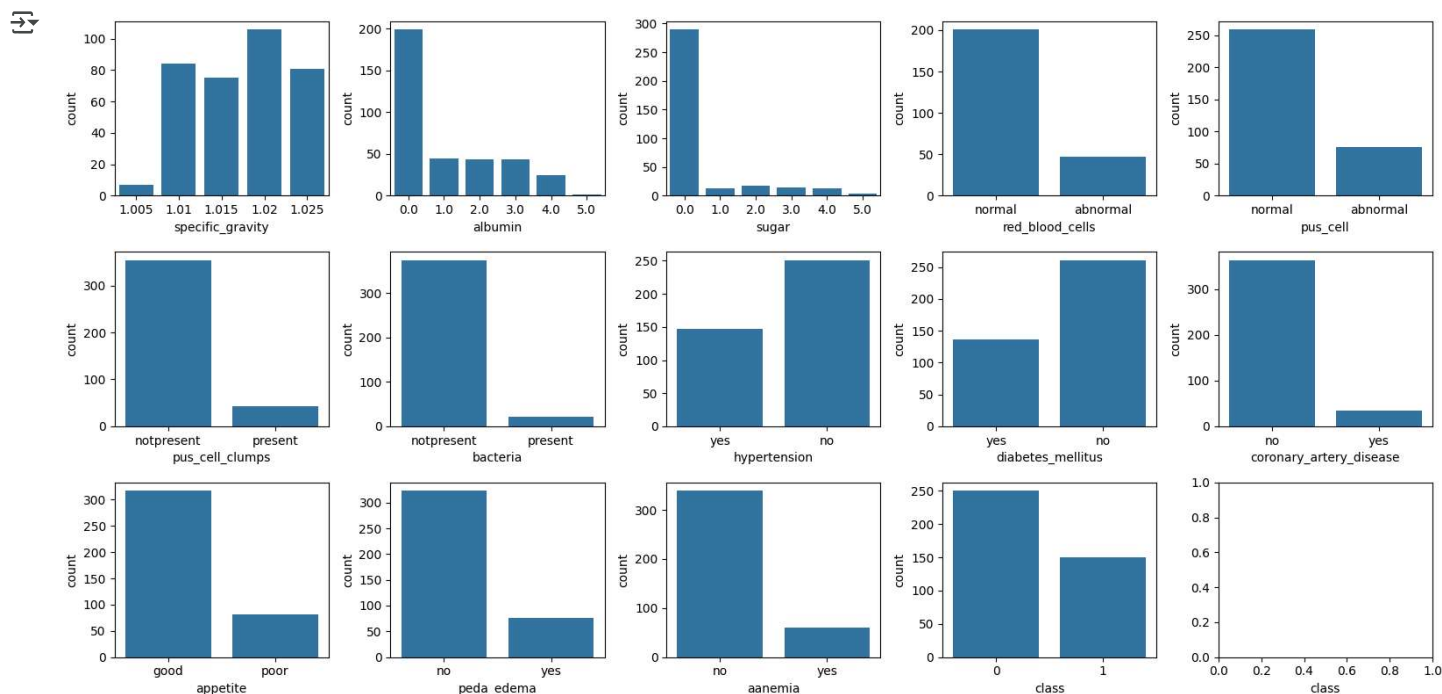
```
# create the figure and axes
```

```
fig, axes = plt.subplots(3, 5, figsize=(16,8))
```

```
axes = axes.ravel() # flattening the array makes indexing easier
```

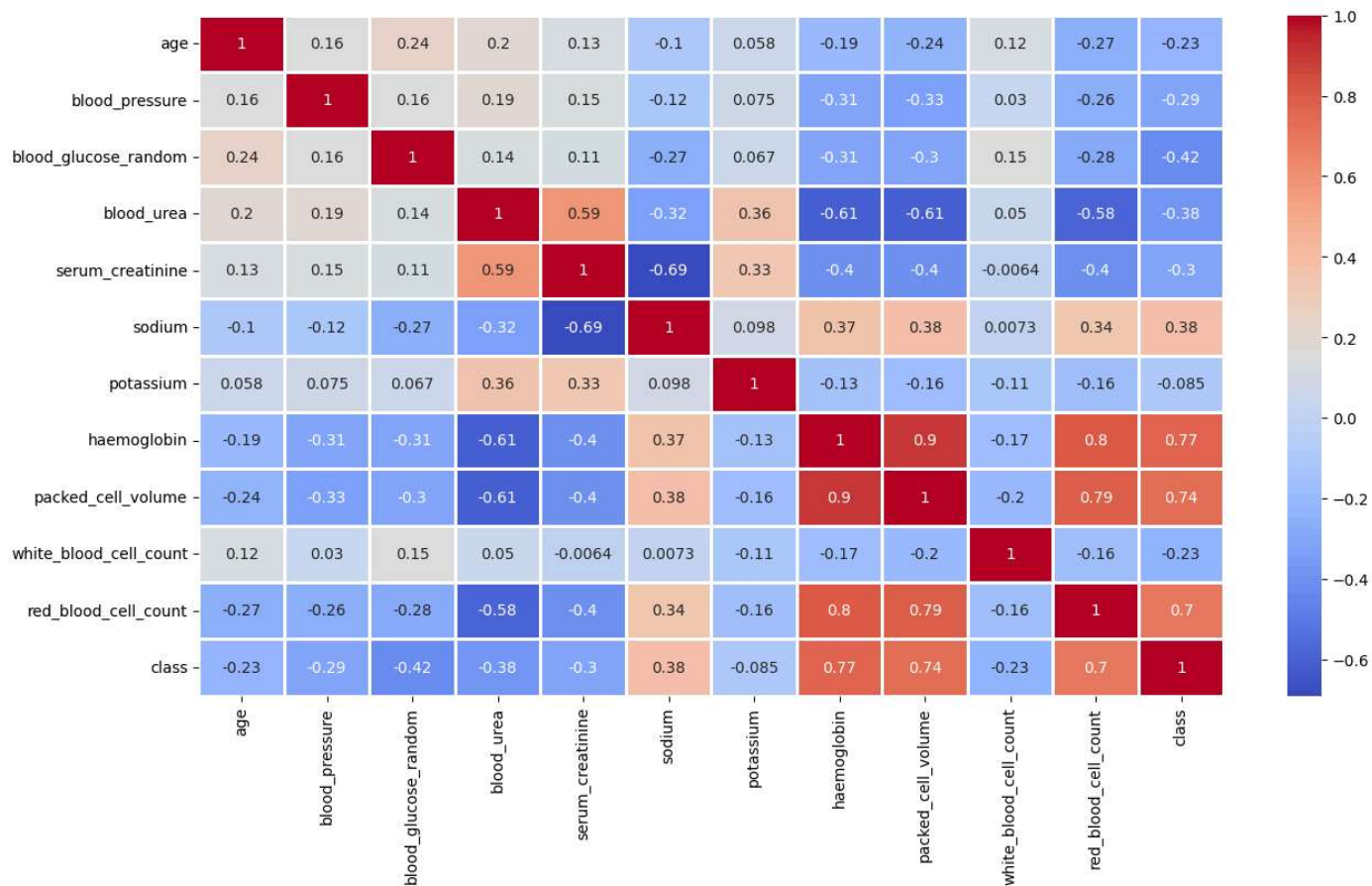
```
# loop over cat cols and plot countplot
```

```
for col,ax in zip(cat_cols,axes):
    sns.countplot(data=chronic_df, x=col, ax=ax)
    plt.xlabel(col)
    fig.tight_layout()
```



```
# correlated heatmap of data
numerical_df = chronic_df.select_dtypes(include=['int', 'float'])

plt.figure(figsize = (15, 8))
sns.heatmap(numerical_df.corr(), annot = True, cmap = 'coolwarm', linewidths=2)
plt.show()
```



```
# let's check count of null values in whole df
```

```
chronic_df.isnull().sum().sort_values(ascending = False)
```



	0
red_blood_cells	152
red_blood_cell_count	131
white_blood_cell_count	106
potassium	88
sodium	87
packed_cell_volume	71
pus_cell	65
haemoglobin	52
sugar	49
specific_gravity	47
albumin	46
blood_glucose_random	44
blood_urea	19
serum_creatinine	17
blood_pressure	12
age	9
pus_cell_clumps	4
bacteria	4
hypertension	2
diabetes_mellitus	2
coronary_artery_disease	2
appetite	1
peda_edema	1
aanemia	1
class	0

dtype: int64

```
# let's check count of null values in num_cols
chronic_df[num_cols].isnull().sum().sort_values(ascending = False)
```



	0
red_blood_cell_count	131
white_blood_cell_count	106
potassium	88
sodium	87
packed_cell_volume	71
haemoglobin	52
blood_glucose_random	44
blood_urea	19
serum_creatinine	17
blood_pressure	12
age	9

dtype: int64

```
# let's check count of null values in cat_cols
chronic_df[cat_cols].isnull().sum().sort_values(ascending = False)
```




	0
red_blood_cells	152
pus_cell	65
sugar	49
specific_gravity	47
albumin	46
pus_cell_clumps	4
bacteria	4
hypertension	2
diabetes_mellitus	2
coronary_artery_disease	2
appetite	1
peda_edema	1
aanemia	1
class	0

dtype: int64

✓ Missing Value Treatment

```
# filling null values, we will use two methods, random sampling for higher null values and
# mean/mode sampling for lower null values
```

```
# creating func for imputing random values
#def random_value_imputation(feature):
#    random_sample =chronic_df[feature].dropna().sample(chronic_df[feature].isnull().sum())
#    random_sample.index =chronic_df[chronic_df[feature].isnull()].index
#    chronic_df.loc[chronic_df[feature].isnull(), feature] =random_sample
```

```
# creating func for imputing most common value(modal value)
#def impute_mode(feature):
#    mode = chronic_df[feature].mode()[0]
#    chronic_df[feature] = chronic_df[feature].fillna(mode)
```

```
# filling num_cols null values using random sampling method
#for col in num_cols:
#    random_value_imputation(col)
```

```
# let's check count of null values in num_cols again
#chronic_df[num_cols].isnull().sum().sort_values(ascending = False)
```

```
# filling "red_blood_cells" and "pus_cell" using random sampling method and rest of cat_cols using mode imputation
#random_value_imputation('red_blood_cells')
#random_value_imputation('pus_cell')
```

```
#for col in cat_cols:
#    impute_mode(col)
```

```
# let's check count of null values in cat_cols again
```

```
# let's check count of null values in cat_cols again
#chronic_df[cat_cols].isnull().sum().sort_values(ascending = False)
```

```
# check unique values in each cat col by looping over cat cols
for col in cat_cols:
    print(f"{col} has {chronic_df[col].nunique()}values\n")
```

```
specific_gravity has 5values
albumin has 6values
sugar has 6values
red_blood_cells has 2values
pus_cell has 2values
pus_cell_clumps has 2values
bacteria has 2values
hypertension has 2values
diabetes_mellitus has 2values
coronary_artery_disease has 2values
appetite has 2values
peda_edema has 2values
aanemia has 2values
class has 2values
```

```
chronic_df.head()
```

```

age  blood_pressure  specific_gravity  albumin  sugar  red_blood_cells  pus_cell  pus_cell_clumps  bacteria  blood_glucose_random .
0  48.0             80.0             1.02      1.0    0.0             NaN      normal      notpresent  notpresent             121.0
1   7.0             50.0             1.02      4.0    0.0             NaN      normal      notpresent  notpresent             NaN
2  62.0             80.0             1.01      2.0    3.0             normal     normal      notpresent  notpresent             423.0
3  48.0             70.0             1.005     4.0    0.0             normal  abnormal      present    notpresent             117.0
4  51.0             80.0             1.01      2.0    0.0             normal     normal      notpresent  notpresent             106.0
5 rows × 25 columns
```

```
# using labelencoder and applying on cat cols
#from sklearn.preprocessing import LabelEncoder

#le = LabelEncoder()

#for col in cat_cols[3:]:
#    chronic_df[col] =le.fit_transform(chronic_df[col])

# check chronic df after transforming cat cols
#chronic_df.head()
```